



KARADENİZ TEKNİK ÜNİVERSİTESİ — BİLGİSAYAR BİLİMLERİ

Hafta 1 Quiz

C'den C++'a Geçiş

Doç. Dr. Tolga Berber · <https://avesis.ktu.edu.tr/tberber>

Öğrenme fazları: 1 • Ön-öğretim 2 • İçerik 3 • Katılım 4 • Değerlendirme 5 • İzleme

Soru 1 (15 puan)

4 • Değerlendirme Hedef 1

C++ programları neden tek adımda değil, **derleme** + **bağlama** olarak iki ayrı aşamada üretilir; header dosyaları bu modelde ne işe yarar?

- (a) Header dosyaları kendi başına derlenip ayrı bir .o nesne dosyası üretir; bu nesne dosyaları daha sonra bağlayıcı tarafından birleştirilir.
- (b) Her .cpp dosyası önışlemciden geçip ayrı ayrı derlenir ve kendi .o nesne dosyasını üretir; header dosyaları yalnızca bildirim taşıyıp önışlemci aşamasında ilgili çeviri birimine kopyalanır, bağlayıcı bu .o dosyalarını birleştirip tek çalıştırılabilir dosyayı oluşturur.
- (c) Derleme aşaması nesne dosyalarını birleştirir, bağlama aşaması ise kaynak kodu makine diline çevirir.
- (d) İki aşamaya ayrılmasının tek nedeni, C++ standardının eski C derleyicileriyle geriye dönük uyumluluk sağlama zorunluluğudur.

Soru 2 (15 puan)

4 • Değerlendirme Hedef 2

<iostream> akışlarını kullanan `std::cin >> sayi;` ifadesi, C'nin `scanf("%d", &sayi);` ifadesine kıyasla hangi temel avantajı sağlar?

- (a) `std::cin >>`, hedef değişkenin tipini kendisi bilir; bu yüzden format dizgesini (%d gibi) elle belirtme zorunluluğu ve buna bağlı tip uyumsuzluğu riski ortadan kalkar. `scanf` ise format dizgesini elle eşleştirmeye dayanır; yanlış belirteç kullanımı tanımsız davranışa yol açabilir.
- (b) `std::cin`, `scanf`'a göre daha hızlı çalışır; bunun nedeni namespace kullanmasıdır.
- (c) `std::cin`, değişkenin adresini otomatik alır ama `scanf` bunu hiçbir zaman yapamaz; bu yüzden `scanf` her koşulda hatalıdır.
- (d) `std::cin` ve `scanf` arasında yalnızca sözdizimsel bir fark vardır; ikisi de aynı düzeyde tip güvenliği sağlar.

Soru 3 (15 puan)

4 • Değerlendirme Hedef 2

Aşağıdakilerden hangisi bir C++ referansı (`int&`) ile bir C işaretçisi arasındaki farkı DOĞRU açıklar?

Soru 3 (devamı)

- (a) Referans kurulduktan sonra başka bir değişkene yeniden bağlanabilir; işaretçi bir kez atandıktan sonra hiçbir zaman değiştirilemez.
- (b) Referans boş (null) olamaz ve kurulduktan sonra başka bir değişkene yeniden bağlanamaz; işaretçi boş olabilir ve başka bir adresi göstermek üzere yeniden atanabilir.
- (c) Referans ve işaretçi tamamen aynı sözdizimini kullanır; ikisi de çağırın tarafta * ile "dereference" edilmelidir.
- (d) İşaretçi yalnızca fonksiyon parametrelerinde kullanılabilir, referans ise yalnızca global değişkenlerde kullanılabilir.

Soru 4 (15 puan)**4 • Değerlendirme Hedef 4**

#pragma once satırı olmasaydı, aynı header'ı hem doğrudan hem de dolaylı yoldan iki kez include eden bir .cpp dosyasında ne olurdu?

- (a) Header içeriği ilgili çeviri birimine iki kez kopyalanır; struct/fonksiyon bildirimleri yeniden tanımlanmış olur ve derleyici "yeniden tanımlama" (*redefinition*) hatası verir.
- (b) Derleyici otomatik olarak ikinci kopyayı fark edip yok sayar; herhangi bir hata oluşmaz.
- (c) Program derlenir, ama yalnızca çalışma zamanında (*runtime*) bir bellek sızıntısına yol açar.
- (d) namespace bloğu otomatik olarak devre dışı kalır ve tüm tanımlar global kapsama taşınır.

Soru 5 (15 puan)**4 • Değerlendirme Hedef 2**

Aşağıdaki kodun ekrana ne bastığını yazınız.

```
1 void ikiyeKatla(int& sayi) {
2     sayi = sayi * 2;
3 }
4
5 int main() {
6     int sayi = 21;
7     ikiyeKatla(sayi);
8     std::cout << sayi;
9 }
```

Soru 6 (15 puan)**4 • Değerlendirme Hedef 3**

Aşağıdaki kodun derlenip derlenmediğini belirtiniz; derlenmiyorsa nedenini kısaca açıklayınız.

```
1 void mesajGoster(const std::string& mesaj) {
2     mesaj = mesaj + "!";
3     std::cout << mesaj;
4 }
```

Soru 7 (10 puan)**4 • Değerlendirme** Hedef 3

Aşağıdaki kodun ekrana ne bastığını yazınız; `toplam` değişkeninin tipinin derleyici tarafından nasıl belirlendiğini kısaca açıklayınız.

```
1 auto a = 7;
2 auto b = 2.5;
3 auto toplam = a + b;
4 std::cout << toplam;
```

Soru 8 (15 puan)**4 • Değerlendirme** Hedef 1

C'de bir `struct`'ın alanlarını, kod tabanındaki herhangi bir fonksiyon doğrudan (ve hatalı bir değerle) değiştirebilir. C++'ın bu soruna karşı getirdiği yaklaşımı — derleme modeli, tip güvenliği ve `const` gibi araçlar açısından — kendi cümlelerinizle açıklayınız.

Soru 9 (15 puan)**4 • Değerlendirme** Hedef 2

Bir referans (`int&`) ile bir işaretçi (`int*`) arasındaki *en az iki* temel farkı yazınız ve bir fonksiyon parametresinde hangisini ne zaman tercih edeceğinizi bir örnekle açıklayınız.
